Unit III

FRONT END TECHNOLOGIES

Front-End Frameworks: What is web framework? Why Web Framework? Web Framework Types. MVC: What is MVC, MVC Architecture, MVC in Practical, MVC in Web Frameworks.

TypeScript: Introduction to TypeScript (TS), Variables and Constants, Modules in TS.

AngularVersion 10+: Angular CLI, Angular Architecture, Angular Project Structure, Angular Lifecycle, Angular Modules, Angular Components, Angular Data Binding, Directives and Pipes, Angular Services and Dependency Injections (DI), Angular Routers, Angular Forms.

ReactJS: Introduction to ReactJS, React Components, Inter Components Communication, Components Styling, Routing, Redux- Architecture, Hooks- Basic hooks, useState() hook, useEffect() hook, useContext() hook.

What are Front-End Technologies?

Front-End Technologies are tools, languages, and frameworks used to design and develop the user interface (UI) of a website or web application.

They control:

- Layout
- Colors
- Fonts
- Buttons
- Forms+
- Images
- Animations
- User interaction

Everything a user sees and interacts with in a browser is created using front-end technologies.

A **Web Framework** is a software platform or collection of tools and libraries that helps developers build web applications easily and efficiently.

It provides:

- Pre-written code structure
- Reusable components
- Standardized development pattern
- Built-in tools for common tasks

Instead of writing everything from scratch, developers use frameworks to speed up development and maintain clean, organized code.

Why Web Framework

1. Faster Development

- Pre-built functions
- Reusable components
- Less coding from scratch

2. Organized Code Structure

- Follows MVC or component-based architecture
- Easy to understand and maintain

3. Reusability

- Components can be reused multiple times

4. Security

- Built-in protection against:
    - XSS (Cross-Site Scripting)
    - CSRF (Cross-Site Request Forgery)
    - SQL Injection (backend frameworks)

5. Easy Maintenance

- Clean structure
- Separation of concerns

6. Scalability

- Suitable for small to large applications

Web Framework Types

Web frameworks are mainly divided into:

1) Front-End Frameworks (Client-Side)
2) Back-End Frameworks (Server-Side)
3) Full-Stack Frameworks

1. Front-End Frameworks (Client-Side Frameworks)

These frameworks are used to design and manage the user interface (UI).

They run in the browser and control:

- Layout
- Buttons
- Forms
- Dynamic content
- Animations

Examples:

- React
- Angular
- Vue.js
- Bootstrap

Features:

- Component-based architecture
- Single Page Applications (SPA)
- Fast rendering

2. Back-End Frameworks (Server-Side Frameworks)

These frameworks manage:

- Server logic
- Database operations
- Authentication
- APIs

They work on the server.

Examples:

- Django
- Laravel
- Express.js
- Spring Boot

Features:

- Database connectivity
- Security features
- REST API creation

3. Full-Stack Frameworks

These frameworks handle both:

- Front-End
- Back-End

Examples:

- Next.js
- Ruby on Rails
- Meteor

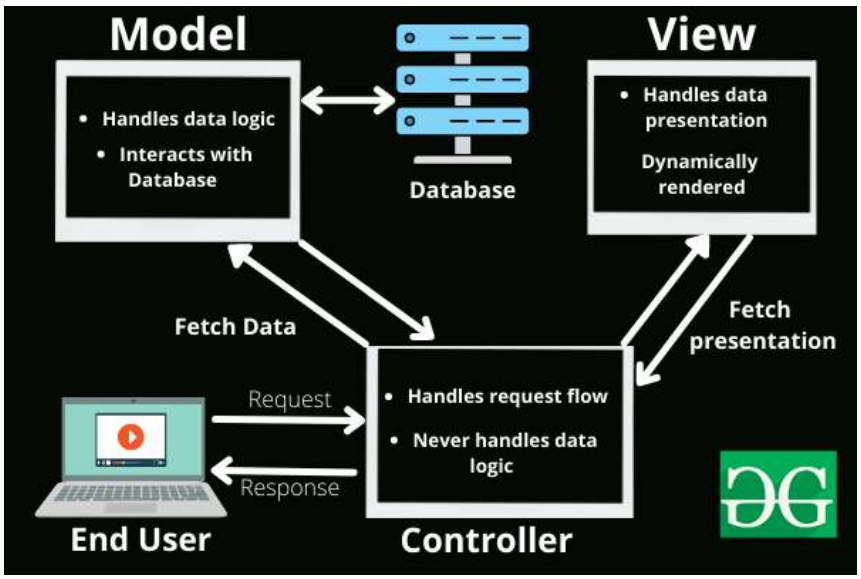
MVC

- The Model-View-Controller (MVC) framework is an architectural/design pattern that separates an application into three main logical components Model, View, and Controller.
- Each architectural component is built to handle specific development aspects of an application. It isolates the business logic and presentation layer from each other. It was traditionally used for desktop graphical user interfaces (GUIs).
- Nowadays, MVC is one of the most frequently used industry-standard web development frameworks to create scalable and extensible projects. It is also used for designing mobile apps.

Components of MVC

The MVC framework includes the following 3 components:

- Controller
- Model
- View



Controller

The controller is the component that enables the interconnection between the views and the model so it acts as an intermediary. The controller doesn't have to worry about handling data logic, it just tells the model what to do. It processes all the business logic and incoming requests, manipulates data using the **Model** component, and interact with the **View** to render the final output.

Responsibilities:

- Receiving user input and interpreting it.
- Updating the Model based on user actions.
- Selecting and displaying the appropriate View.

Example: In a bookstore application, the Controller would handle actions such as searching for a book, adding a book to the cart, or checking out.

View

The **View** component is used for all the UI logic of the application. It generates a user interface for the user. Views are created by the data which is collected by the model component but these data aren’t taken directly but through the controller. It only interacts with the controller.

Responsibilities:

- Rendering data to the user in a specific format.
- Displaying the user interface elements.
- Updating the display when the Model changes.

Example: In a bookstore application, the View would display the list of books, book details, and provide input fields for searching or filtering books.

Model

The **Model** component corresponds to all the data-related logic that the user works with. This can represent either the data that is being transferred between the View and Controller components or any other business logic-related data. It can add or retrieve data from the database. It responds to the controller's request because the controller can't interact with the database by itself. The model interacts with the database and gives the required data back to the controller.

Responsibilities:

- Managing data: CRUD (Create, Read, Update, Delete) operations.
- Enforcing business rules.
- Notifying the View and Controller of state changes.

Example: In a bookstore application, the Model would handle data related to books, such as the book title, author, price, and stock level.

MVC Architecture Flow (Step-by-Step)

How MVC Works:

1. User sends request (click button)
2. Controller receives request
3. Controller calls Model
4. Model interacts with database

5. Model sends data to Controller
6. Controller sends data to View
7. View displays result to user

Example: Login System

Step 1: User enters username & password

(View – Login Form)

Step 2: Clicks Login button

(Controller receives request)

Step 3: Controller sends data to Model

Model checks database

Step 4: Model returns result

Valid or Invalid

Step 5: Controller sends response to View

View shows:

- "Login Successful" OR
- "Invalid Credentials"

Features of MVC

- It provides a clear separation of business logic, UI logic, and input logic.
- It offers full control over your HTML and URLs which makes it easy to design web application architecture.
- It is a powerful URL-mapping component using which we can build applications that have comprehensible and searchable URLs.
- It supports Test Driven Development (TDD).

Advantages of MVC

- Codes are easy to maintain and they can be extended easily.
- The MVC model component can be tested separately.
- The components of MVC can be developed simultaneously.

- It reduces complexity by dividing an application into three units. Model, view, and controller.
- It supports Test Driven Development (TDD).
- It works well for Web apps that are supported by large teams of web designers and developers.
- This architecture helps to test components independently as all classes and objects are independent of each other
- Search Engine Optimization (SEO) Friendly.

Disadvantages of MVC

- It is difficult to read, change, test, and reuse this model
- It is not suitable for building small applications.
- The inefficiency of data access in view.
- The framework navigation can be complex as it introduces new layers of abstraction which requires users to adapt to the decomposition criteria of MVC.
- Increased complexity and Inefficiency of data

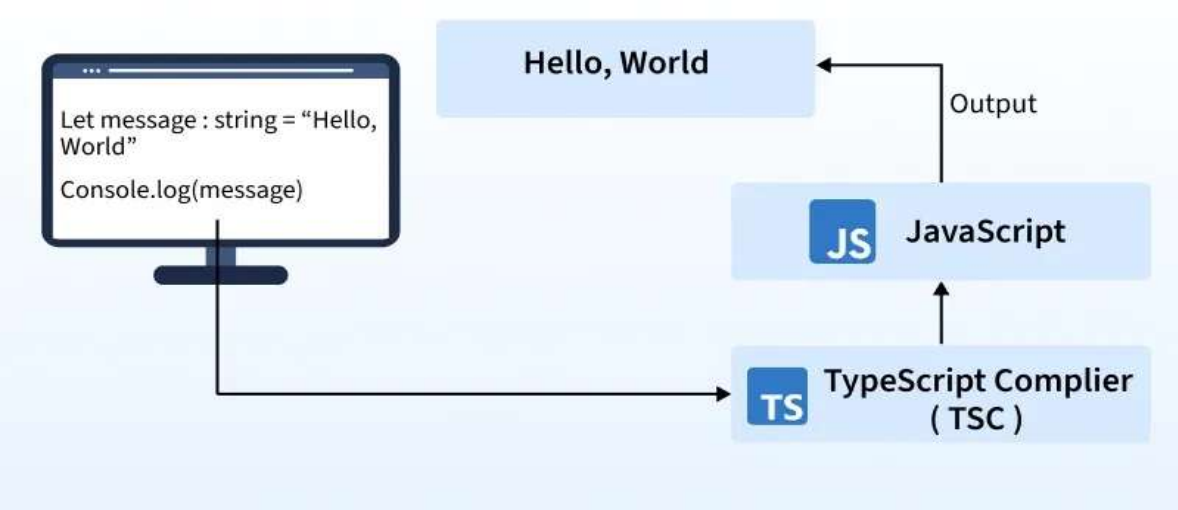
Popular MVC Frameworks

Some of the most popular and extensively used MVC frameworks are listed below.

- Ruby on Rails
- Django
- CherryPy
- Spring MVC
- Catalyst
- Rails
- Zend Framework
- Fuel PHP
- Laravel
- Symphony

TypeScript

- **TypeScript** is a statically typed superset of JavaScript that compiles to plain JavaScript. It adds features like static typing, interfaces, generics, and modern ECMAScript support, helping developers catch errors early and build scalable applications with better tooling such as autocompletion, refactoring, and debugging.
- TypeScript code is converted (compiled) into JavaScript using the TypeScript compiler (tsc).



JavaScript Vs TypeScript

Here are the detailed difference between JavaScript and TypeScript:

JavaScript	TypeScript
A lightweight, interpreted scripting language used for web development.	A superset of JavaScript that adds static typing and advanced features.

JavaScript	TypeScript
Dynamically typed (types are checked at runtime).	Statically typed (optional) – types are checked at compile time.
Runs directly in browsers or Node.js (no compilation needed).	Must be compiled to JavaScript using the TypeScript compiler (tsc).
Errors appear only at runtime.	Errors are caught at compile time, reducing runtime bugs.
Prototype-based object-oriented programming.	Class-based object-oriented programming with interfaces, access modifiers, generics.
Supports ES6+ features depending on the environment.	Supports all modern ES6+ features, plus extra TypeScript-only features.
Basic editor support, limited IntelliSense.	Rich tooling with autocompletion, type checking, and refactoring support.
Runs everywhere (browser, Node.js, servers).	Used in large-scale applications, compiled to JS for execution.

Features of TypeScript

1. Static Typing
2. Object-Oriented Programming (OOP) support

3. Interfaces
4. Generics
5. Modules
6. Better IDE support (auto-completion, error detection)

TypeScript Works

TypeScript Code (.ts)

↓

TypeScript Compiler (tsc)

↓

JavaScript Code (.js)

↓

Browser / Node.js

Variables and Constants in TypeScript

TypeScript supports three ways to declare variables:

- var (old, function-scoped)
- let (block-scoped)
- const (block-scoped constant)

Variable Declaration with Type

Syntax:

- let variableName: dataType = value;

Example

```
let name: string = "Shraddha";
let age: number = 22;
let isStudent: boolean = true;
```

Common Data Types in TypeScript

Type	Example
String	"Hello"
number	100
boolean	True
Any	can store any type
Void	function returns nothing
Null	null value
undefined	undefined value
Array	number[]
Tuple	[string, number]

Example:

```
let numbers: number[] = [10, 20, 30];
let person: [string, number] = ["Shraddha", 22];
```

Using const

Used when value should not change.

```
const pi: number = 3.14;
```

Cannot reassign:

```
pi = 3.1415; // Error
```

Modules in TypeScript

- A **Module** in TypeScript is a file that contains code (variables, functions, classes, interfaces, etc.) and **exports** them so they can be reused in other files.
- Modules help divide your application into **small reusable parts**.
- Every TypeScript file becomes a **module** if it contains:
- If there is no **export/import** → it is treated as a normal script

Advantages of Modules

- Code reusability
- Better structure
- Easier debugging
- Supports large applications

AngularVersion 10+

- Angular is a TypeScript-based front-end web application framework developed by Google.
- It is used for building single-page applications (SPAs) and dynamic web apps.
- Angular provides a structured way to develop applications by following a component-based architecture, making it easy to manage and scale.
- It Supports MVC architecture
- Provides built-in tools for:
    - Routing
    - Forms
    - HTTP services
    - Dependency Injection

Features of Angular 10+

Angular 10+ introduced improvements like:

- Faster compilation
- Better performance
- Improved error handling
- Updated TypeScript support
- Ivy compiler improvements

Key Features of AngularVersion 10+:

1. **Component-Based Architecture** – Applications are built using reusable and modular components.
2. **Two-Way Data Binding** – Automatically synchronizes data between the model and the view.
3. **Dependency Injection (DI)** – Manages dependencies efficiently for better code organization.
4. **Directives** – Custom HTML attributes to extend functionality (e.g., *ngIf, *ngFor).
5. **Routing** – Built-in router for navigation between views in SPAs.
6. **Services & HTTP Client** – Fetch and manipulate data from APIs.
7. **RxJS (Reactive Programming)** – Handles asynchronous operations efficiently.

CLI (Command Line Interface)

Angular CLI is a **command-line tool** used to create, develop, test, and build Angular applications easily.

Angular CLI is officially maintained by **Google** as part of the Angular framework. Used to create

- Create Angular projects
- Generate components, services, modules, etc.
- Run development server
- Build and test applications
- Manage configurations

It simplifies Angular development by automating most tasks.

Why Angular CLI is Important?

Before CLI:

- Developers manually created folder structure
- Manually configured Webpack, TypeScript, testing setup

With CLI:

- Automatic project structure
- Built-in development server
- Production build optimization
- Testing support
- Code scaffolding (generate components, services etc.)

Installing Angular CLI

Step 1: Install Node.js

Angular requires Node.js installed.

Step 2: Install CLI globally

npm install -g @angular/cli

Step 3: Check Version

ng version

Creating a New Angular Project

ng new my-app

It will ask:

- Add routing? (Yes/No)
- CSS format? (CSS/SCSS/LESS)

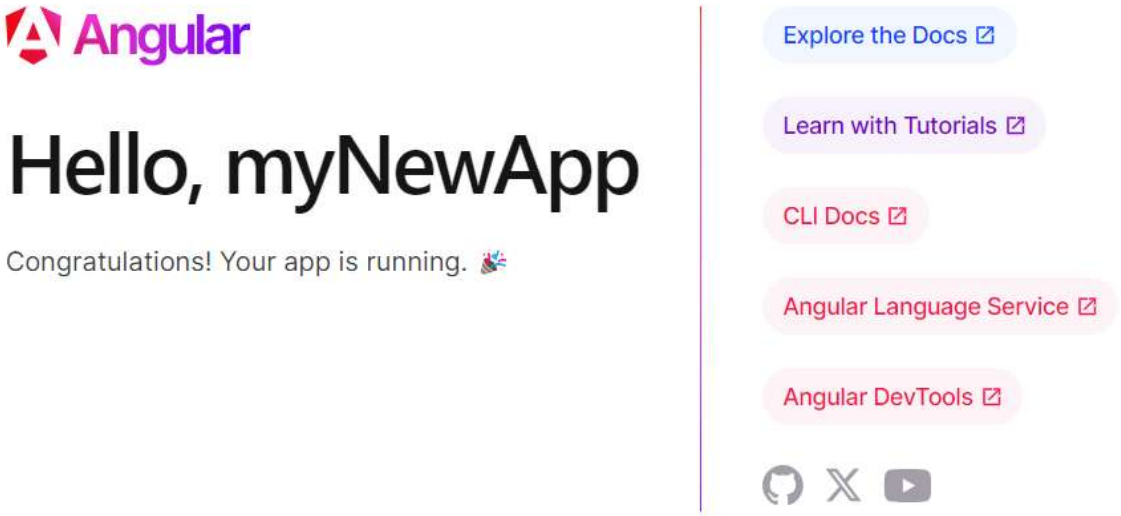
Then run:

cd my-app
ng serve

Open in browser:

http://localhost:4200

Output



Angular CLI Project Structure (Angular 10+)

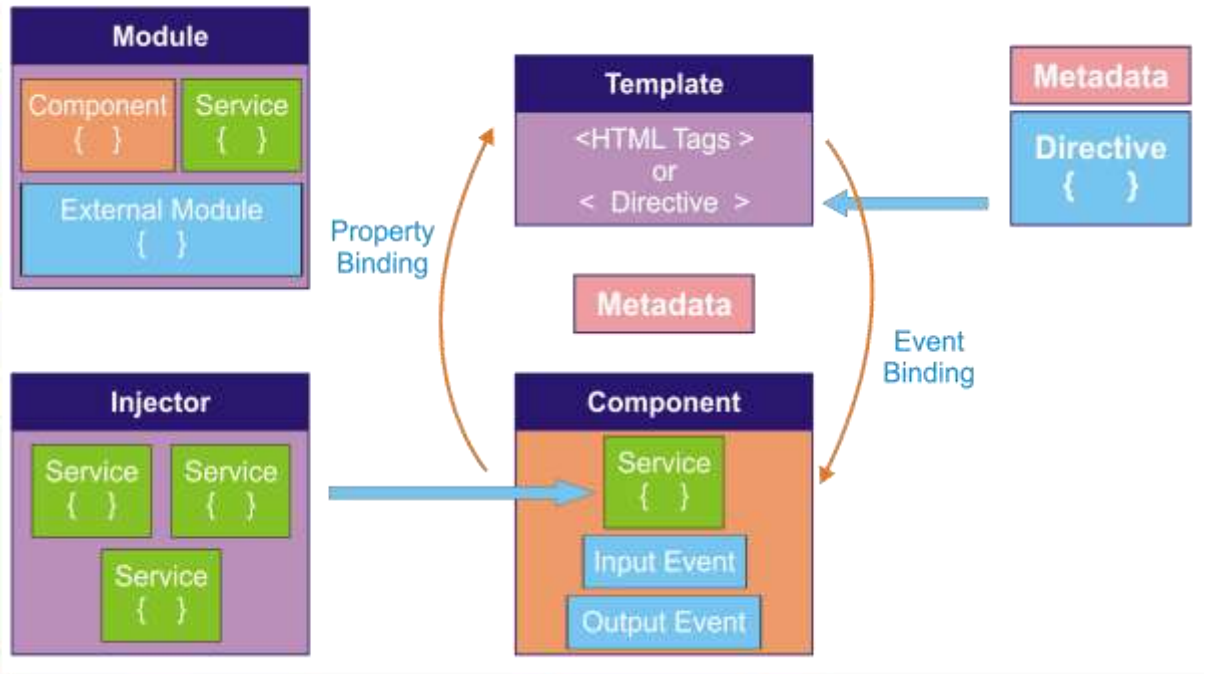
After creating project:

```
my-app/
|
├── src/
|   ├── app/
|   |   ├── app.component.ts
|   |   ├── app.module.ts
|   |
|   ├── assets/
|   ├── environments/
|
├── angular.json
├── package.json
├── tsconfig.json
```

Angular Architecture

To develop any web application, Angular follows the MVC (Model-View-Controller) and MVVM (Model-View-ViewModel) design patterns, which facilitates a structured and organized approach to designing the application,

along with making it easier to manage code, improve maintainability, & etc. These types of design patterns usually maintain a clear distinction between data (Model), user interface (View), and logic (Controller/ViewModel), which results in more scalable and testable applications. It also provides code reusability and ensures a smoother development process for large & complex projects.



An Angular Application contains the following building blocks:

- Modules
- Components
- Templates
- Directives
- Services
- Dependency Injection(DI)
- Property Binding
- Data Binding

Modules

- A Module is a unit that consists of a separate block to perform specific functionality and enables us to break down the application into smaller chunks.
- In a module, we can export & import the components and services from other modules.
- Modules are created using the *@NgModule* decorator.

- **Types of Modules:**
  - **Core Module/Root Module**
    - Every Angular application must have at least one root module, which is called the **AppModule**, defined in the *app.module.ts* file.
    - The root module is the top-level module in an Angular application.
    - It imports all of the other modules in the application.
  - **Feature Module**
    - Feature modules are used to group related components, directives, pipes, and services together.
  - **Shared Module**
    - The most commonly used functionality will be present in the shared module which can be imported by the feature module whenever needed.

It groups:

- Components
- Services
- External modules

Purpose:

- Organizes application
- Manages dependencies
- Bootstraps main component

Services

- Services are used when specific data or logic needs to be used across different components.
- Services are typically used to centralize data access, HTTP requests, state management, and other common tasks.

- Services are singleton and are registered with Angular's dependency injection system.
- Components can inject services to access their functionality and data.

Templates

Template = View Layer

- Written in HTML
- Can use Angular Directives
- Displays data from component

It contains:

<HTML Tags>
or
<Directive>

Template shows data to user.

Directives

- Directives are instructions in the DOM (Document Object Model).
- Directives are used in templates to customize the behaviour of the elements.
- Angular provides built-in directives like *ngIf and *ngFor, as well as custom directives created by developers.
- **Types of directives**:
    - **Component Directives**
        - These directives are associated with the template(view) of a component.
    - **Structural Directives**
        - These directives are used to change the structure of the DOM using *ngFor,*ngSwitch and *ngIf.
    - **Attribute Directives**
        - These directives are used to change the behaviour of the DOM using ngStyle,ngModel and ngClass.
    - **Custom Directives**

- We can create custom directives using *@Directive* decorator and define the desired behaviour in the class.

Components

- A Component is the building block of the angular application.
- **Component = Heart of Angular**
- A Component controls a part of the UI.

It contains:

- Service (for logic)
- Input Event
- Output Event
  - **Input Event**
- Receives data from parent component
- Uses @Input()
  - **Output Event**
- Sends data to parent component
- Uses @Output()
- Components handle UI logic.

Metadata

Metadata tells Angular:

- How to process a class
- How a component should behave

Examples:

- @Component
- @NgModule
- @Injectable

Metadata connects component to template

Dependency Injection(DI)

Injector = Dependency Injection System

Injector provides Services to components.

Instead of creating object manually:

Angular automatically injects the service.

This is called Dependency Injection (DI).

Property Binding

Property Binding is a technique in Angular used to send data
from the Component (TypeScript class) to the Template (HTML view).

Why We Use Property Binding?

To dynamically set:

- HTML element properties
- DOM properties
- Component properties
- Directive properties

Instead of writing static values, we bind dynamic values from the component.

Event Binding

Event Binding is used to send data
from the Template (HTML view) to the Component (TypeScript class)
when a user performs an action.

Why We Use Event Binding?

To handle user actions like:

- Button click
- Key press
- Mouse hover
- Form submit
- Input change

It allows the component to respond to user events.

Syntax of Event Binding

(event)="method()"

- Round brackets ( ) are used.
- Calls a method defined in the component.

Example: Button Click Event

Component (TypeScript)

```
export class AppComponent {
  showMessage() {
    alert("Button Clicked!");
  }
}
```

Template (HTML)

```
<button (click)="showMessage()">Click Me</button>
```

When user clicks button → method runs.

Complete Flow

1. Module loads component.
2. Component connects to template.
3. Template displays data (Property Binding).
4. User performs action (Event Binding).
5. Component calls Service.
6. Injector provides Service instance.
7. Service handles business logic / API.

Angular Lifecycle

In Angular, every component goes through a series of stages from creation to destruction. These stages are called the Component Lifecycle.

Lifecycle Phases

Angular component lifecycle can be divided into **4 main phases**:

1. Creation Phase
2. Change Detection Phase
3. Rendering Phase
4. Destruction Phase

Angular provides special methods called **Lifecycle Hooks** that allow us to perform actions at specific stages of a component's life.

In your components, you can implement **lifecycle hooks** to run code during these steps. Lifecycle hooks that relate to a specific component instance are implemented as methods on your component class. Lifecycle hooks that relate the Angular application as a whole are implemented as functions that accept a callback.

Complete Lifecycle Flow

constructor()
↓
ngOnChanges()
↓
ngOnInit()
↓
ngDoCheck()
↓
ngAfterContentInit()
↓
ngAfterContentChecked()
↓
ngAfterViewInit()
↓
ngAfterViewChecked()
↓
(Repeated change detection cycle)
↓
ngOnDestroy()

1) constructor()

- Called when component class is instantiated
- Used for dependency injection
- Not a lifecycle hook (TypeScript feature)

```
constructor(private service: DataService) {
  console.log("Constructor Called");
}
```

Avoid heavy logic here.

2) ngOnChanges()

- Triggered when @Input() values change
- Called before ngOnInit()
- Receives SimpleChanges object

```
ngOnChanges(changes: SimpleChanges)
 {
  console.log("Changes:", changes);
}
```

Mostly used in Parent–Child communication.

3) ngOnInit()

- Called once after first ngOnChanges()
- Best place for initialization logic
- Used for API calls

```
ngOnInit() {
  console.log("Component Initialized");
}
```

4) ngDoCheck()

- Runs during every change detection cycle
- Used for manual change detection

```
ngDoCheck() {
  console.log("Change detection running");
}
```

Avoid heavy logic (called many times).

5) ngAfterContentInit()

- Called once after <ng-content> projection
- Used to access projected content

```
ngAfterContentInit() {
  console.log("Content Initialized");
}
```

6) ngAfterContentChecked()

- Called after projected content is checked
- Runs multiple times

```
ngAfterContentChecked() {
  console.log("Content Checked");
}
```

7) ngAfterViewInit()

- Called once after component view & child views initialized
- Used with @ViewChild()

```
ngAfterViewInit() {
  console.log("View Initialized");
}
```

8) ngAfterViewChecked()

- Called after component view is checked
- Runs multiple times

```
ngAfterViewChecked() {
  console.log("View Checked");
}
```

9) ngOnDestroy()

- Called before component is destroyed
- Used for cleanup

```
ngOnDestroy() {
  console.log("Component Destroyed");
}
```

It is important for:

- Unsubscribing Observables
- Clearing intervals
- Removing event listeners

Important Points

- ngOnInit() runs only once
- ngOnChanges() runs before ngOnInit()
- ngDoCheck() runs multiple times
- ngOnDestroy() prevents memory leaks
- Constructor is not a lifecycle hook

ReactJS

ReactJS is a component-based JavaScript library used to build dynamic and interactive user interfaces. It simplifies the creation of single-page applications (SPAs) with a focus on performance and maintainability.

"Hello, World!" Program in React

```
import React from 'react';

function App() {
  return (
    <div>
      <h1>Hello, World!</h1>
    </div>
  );
}

export default App;
```

In this example:

- import React from 'react': Imports React to create components and use JSX.
- function App() { ... }: Defines a functional component called App.
- return (...): Returns JSX that represents the UI (a div with an h1 tag displaying "Hello, World!").
- export default App: Exports the App component so it can be used elsewhere.

Working of React

React operates by creating an in-memory virtual DOM rather than directly manipulating the browser's DOM. It performs necessary manipulations within this virtual representation before applying changes to the actual browser DOM.

1. Actual DOM and Virtual DOM

- Initially, there is an Actual DOM(Real DOM) containing a div with two child elements: h1 and h2.
- React maintains a previous Virtual DOM to track the UI state before any updates.

2. Detecting Changes

- When a change occurs (e.g., adding a new h3 element), React generates a New Virtual DOM.
- React compares the previous Virtual DOM with the New Virtual DOM using a process called reconciliation.

3. Efficient DOM Update

- React identifies the differences (in this case, the new h3 element).
- Instead of updating the entire DOM, React updates only the changed part in the New Actual DOM, making the update process more efficient.

Features of React

React is one of the most demanding JavaScript libraries because it is equipped with a ton of features which makes it faster and production-ready. Below are the few features of React.

1. Virtual DOM

React uses a Virtual DOM to optimize UI rendering. Instead of updating the entire real DOM directly, React:

- Creates a lightweight copy of the DOM (Virtual DOM).
- Compares it with the previous version to detect changes (diffing).
- Updates only the changed parts in the actual DOM (reconciliation), improving performance.

2. Component-Based Architecture

React follows a component-based approach, where the UI is broken down into reusable components. These components:

- Can be functional or class-based.
- It allows code reusability, maintainability, and scalability.

3. JSX (JavaScript XML)

React uses **JSX**, a syntax extension that allows developers to write HTML inside JavaScript. JSX makes the code:

- More readable and expressive.
- Easier to understand and debug.

4. One-Way Data Binding

React uses one-way data binding, meaning data flows in a single direction from parent components to child components via props. This provides better control over data and helps maintain predictable behavior.

5. State Management

React manages component state efficiently using the useState hook (for functional components) or this.state (for class components). State allows dynamic updates without reloading the page.

6. React Hooks

Hooks allow functional components to use state and lifecycle features without needing class components. Common hooks include:

- **useState:** for managing local state.
- **useEffect:** for handling side effects like API calls.
- **useContext:** for global state management.

7. React Router

React provides React Router for managing navigation in single-page applications (SPAs). It enables dynamic routing without requiring full-page reloads.

ReactJS Component

- React components are independent, reusable building blocks in a React application that define what gets displayed on the UI.
- They accept inputs called props and return React elements describing the UI.
- Components can be reused across different parts of the application to maintain consistency and reduce code duplication.
- Components manage how their output is rendered in the DOM based on their state and props.

Types of ReactJS Components

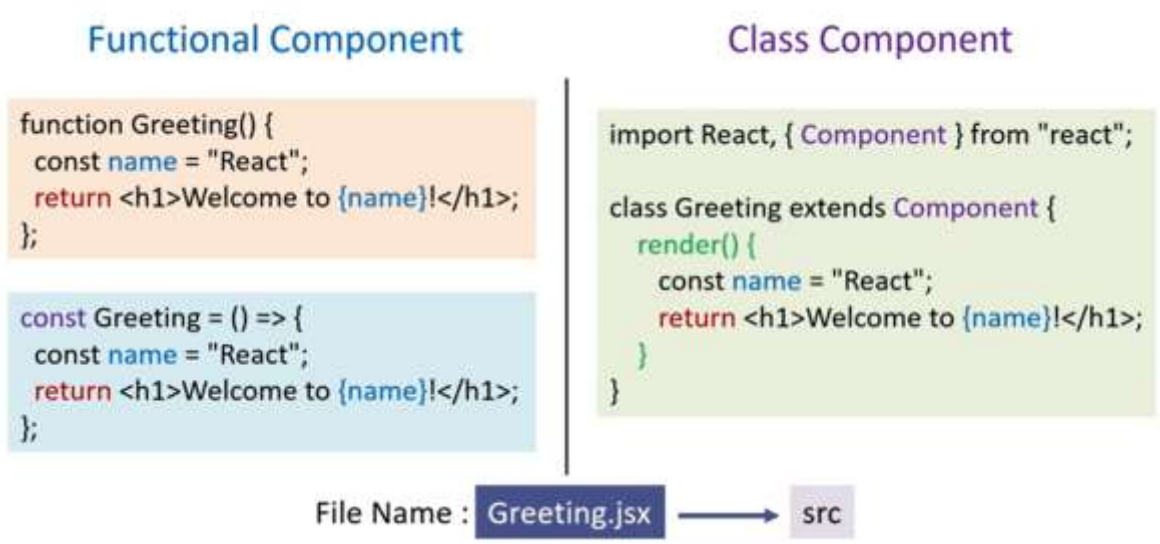
1. **Functional Components**

- Functional components are simpler and preferred for most use cases. They are JavaScript functions that return React elements. With the introduction of React Hooks, functional components can also manage state and lifecycle events.
    - **Stateless or Stateful**: Can manage state using React Hooks.
    - **Simpler Syntax**: Ideal for small and reusable components.
    - **Performance**: Generally faster since they don't require a 'this' keyword.

2. Class Components

Class components are ES6 classes that extend React.Component. They include additional features like state management and lifecycle methods.

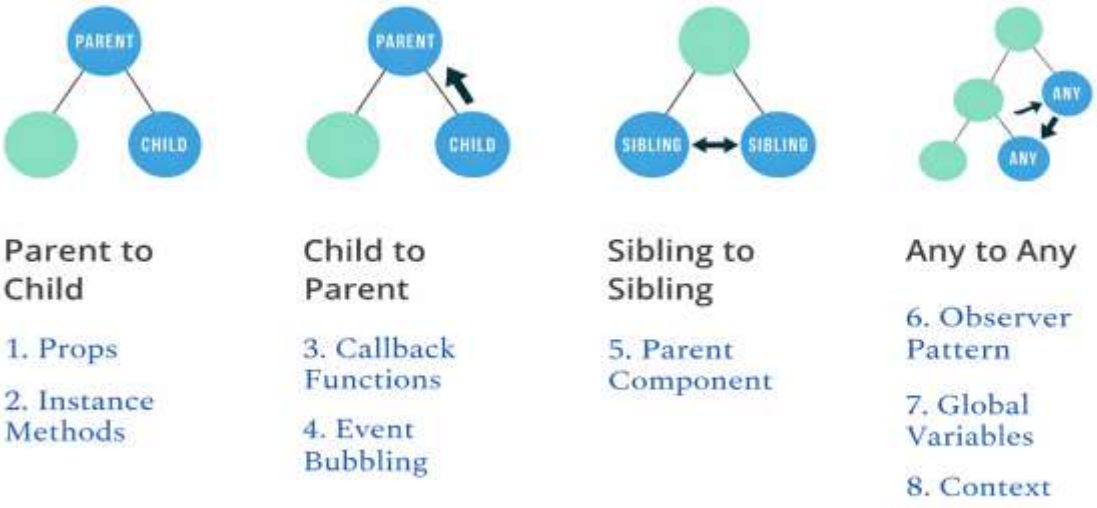
- **State Management:** State is managed using the this.state property.
- **Lifecycle Methods:** Includes methods like componentDidMount, componentDidUpdate, etc.



Inter Components Communication in ReactJS

In React, applications are built using components, and components often need to share data with each other.

React follows unidirectional data flow (Parent → Child).



Types of Communication

1. **Parent → Child Communication**

- Data flows **downward** from Parent component to Child component.
- Parent sends data
- Child receives data (read-only)

Method 1: Props

Props = properties used to pass data.

Flow

Parent → Child

Example

Parent

```
function Parent() {
  return <Child name="John" />;
}
```

Child

```
function Child(props) {
  return <h1>Hello {props.name}</h1>;
}
```

Method 2: Instance Methods (Using refs)

Parent directly calls a child's function using **ref**.

Example

```
import { useRef } from "react";

function Parent() {
  const childRef = useRef();

  return (
    <>
      <Child ref={childRef} />
      <button onClick={() => childRef.current.show()}>
        Call Child Method
      </button>
    </>
  );
}
```

2. **Child → Parent Communication**

React does NOT allow direct upward data flow.

So we use **functions passed from parent**.

Method 3: Callback Functions

- Parent sends a function → Child calls it.
- Most used method for child → parent

Flow

Parent (function)
↓
Child executes function
↓
Parent receives data

Example

Parent

```
function Parent() {
  const receiveData = (msg) => {
    console.log(msg);
  };

  return <Child sendData={receiveData} />;
}
```

Child

```
function Child(props) {
  return (
    <button onClick={() => props.sendData("Hello Parent")}>
      Send Data
    </button>
  );
}
```

Method 4: Event Bubbling

- Events automatically move upward in React.
- Clicking button triggers parent handler.
- Useful for handling UI events.

Example:

```
<div onClick={handleParent}>
  <button>Click</button>
</div>
```

3. Sibling → Sibling Communication

- Sibling components cannot communicate directly.
- They use a Common Parent.
- Parent stores shared state.

Flow

Sibling A → Parent → Sibling B

Example

Parent

```
function Parent() {
  const [data, setData] = useState("");

  return (
    <>
      <ChildA send={setData} />
      <ChildB value={data} />
    </>
  );
}
```

4 Any → Any Communication

When many components need shared data.

Example:

- Login user info
- Theme (dark/light)
- Language settings

Method 6: Observer Pattern

Components subscribe to changes.

Example:

- Redux store
- Event emitters

State changes → All subscribers update automatically

Method 7: Global Variables

Data stored globally.

Example:

```
window.username = "John";
```

Not recommended because:

- Hard to manage
- Debugging issues

- Breaks React structure

Method 8: Context API (Best Built-in Method)

Allows sharing data without passing props manually.

Step 1: Create Context

```
import { createContext } from "react";
export const UserContext = createContext();
```

Step 2: Provide Data

```
<UserContext.Provider value="Shraddha">
  <App />
</UserContext.Provider>
```

Step 3: Consume Data

```
import { useContext } from "react";
const user = useContext(UserContext);
```

Avoids **prop drilling**.

Component Styling in ReactJS

React supports multiple styling methods.

1. Inline Styling

- CSS written inside JSX.
- Style is written as JavaScript object.

```
function App() {
 return (
   <h1 style={{ color: "blue", fontSize: "30px" }}>
     Hello React
   </h1>
 );
}
```

2. External CSS File

Create App.css.

```
.title {
  color: red;
}
```

Import CSS**:**

```
import "./App.css";

<h1 className="title">Hello</h1>
```

3. CSS Modules (Scoped Styling)

Avoids style conflicts.

App.module.css

```
.title {
  color: green;
}
```

Component

```
import styles from "./App.module.css";

<h1 className={styles.title}>Hello</h1>
```

4. Styled Components (Library)

Install:

```
npm install styled-components
```

Example:

```
import styled from "styled-components";

const Button = styled.button`
  background: blue;
  color: white;
`;

function App() {
  return <Button>Click</Button>;
}
```

Routing in ReactJS

Routing allows navigation between pages without reloading the website.

Install React Router

npm install react-router-dom

Basic Routing Structure

Step 1: Import Router

```
import { BrowserRouter, Routes, Route } from "react-router-dom";
```

Step 2: Create Components

Home.js

```
function Home() {
  return <h2>Home Page</h2>;
}
export default Home;
```

About.js

```
function About() {
  return <h2>About Page</h2>;
}
export default About;
```

Step 3: Configure Routes

```
import Home from "./Home";
import About from "./About";

function App() {
  return (
    <BrowserRouter>
      <Routes>
        <Route path="/" element={<Home />} />
        <Route path="/about" element={<About />} />
      </Routes>
    </BrowserRouter>
  );
}
```

export default App;

🔹 Navigation Using Link

import { Link } from "react-router-dom";

<Link to="/">Home</Link>
<Link to="/about">About</Link>

Page changes without refresh.

Types of Routing in React

1️⃣ Basic Routing

Simple page navigation.

/home
/about
/contact

2️⃣ Nested Routing

Route inside another route.

Example

```
<Route path="/dashboard" element={<Dashboard />}>
  <Route path="profile" element={<Profile />} />
</Route>
```

URL:

/dashboard/profile

3️⃣ Dynamic Routing (URL Parameters)

Used when data changes dynamically.

Example:

/user/101
/user/102

Route Setup

<Route path="/user/:id" element={<User />} />

Access Parameter

```
import { useParams } from "react-router-dom";

function User() {
  const { id } = useParams();
  return <h2>User ID: {id}</h2>;
}
```

4 Protected Routing (Authentication)

Restricts access without login.

Example

```
function ProtectedRoute({ children }) {
  const isLogin = true;

  return isLogin ? children : <h1>Login Required</h1>;
}
```

Usage:

```
<Route
 path="/dashboard"
 element={
   <ProtectedRoute>
     <Dashboard />
   </ProtectedRoute>
 }
/>
```

5 Redirect Routing

Automatically move user to another page.

```
import { Navigate } from "react-router-dom";

<Route path="*" element={<Navigate to="/" />} />
```

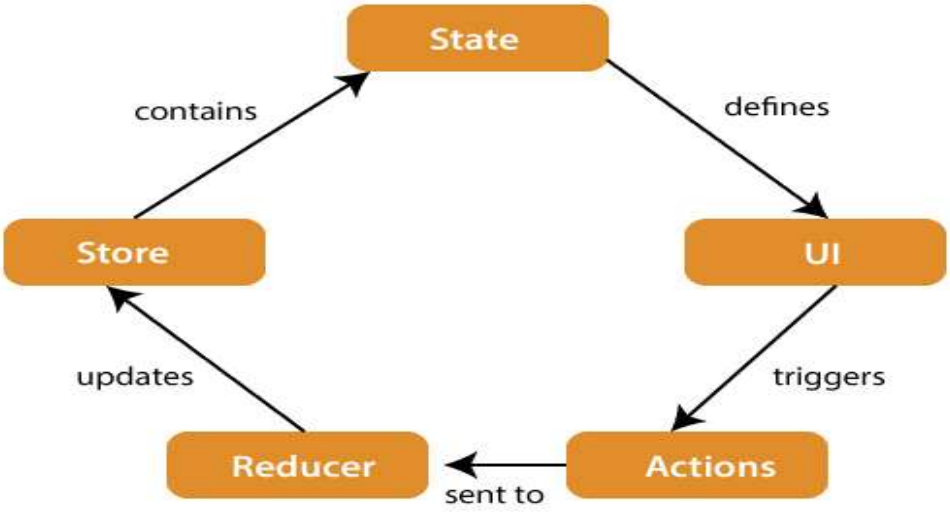
Used for invalid URLs.

Redux Architecture

- Redux is a state management library used mainly with React applications to manage global application state in a predictable way.

- It stores all application data in one central store.

Redux Architecture



1)State

- **State** contains the **application data**.
- It represents the **current condition** of the app.

2)UI (User Interface)

- UI displays data from the **state**.
- React components read data and show it on screen.

3)Action

- Actions are plain JavaScript objects describing what happened**.**

4)Reducer

- Reducer is a function that updates state based on actions.
- Reducers are **pure** functions**.**

5)Store

- The Store is the single source of truth that holds the entire application state.
- Only one store exists in Redux.
- Holds application state

- Allows state access
- Updates state using reducers

Flow of Architecture

- **User Interaction/Event**: Something happens in the application (e.g., a form submission, button click).
- **Dispatch Action**: The UI code dispatches an action object to the Redux store.
- **Store & Reducer Execution**: The store runs the root reducer function, passing the *previous state* and the *current action*. The reducer calculates the new state.
- **State Update**: The store saves the return value from the reducer as the new application state.
- **UI Update**: The store notifies all subscribed UI components that the state has been updated. Components check if the data they need has changed and re-render with the new data.

Advantages of Redux

- Predictable state management
- Easy debugging
- Centralized dataTime-travel debugging
- Scalable applications

Hooks in ReactJS

Hooks are special functions in React that allow functional components to use:

- State
- Lifecycle features
- Context

Before Hooks, these features were only available in class components.

Why Hooks are Used?

Earlier (Class Components):

- Complex syntax
- this keyword confusion

- Hard to reuse logic

  Hooks provide:

  ✓ Simpler code
  ✓ Reusable logic
  ✓ Better readability
  ✓ Functional programming style

Types of React Hooks

1) Basic Hook
2) Advance Hook
3) Custom Hook

Rules of Hooks

- Use hooks only inside React functions
- Call hooks at the top level (not inside loops/conditions)
- Only call hooks in React components or custom hooks

The three basic hooks for fundamental use cases are useState(), useEffect(), and useContext()

1) **useState()**
   - It is used to create and manage state inside functional components.
   - It takes an initial state value as an argument and returns an array containing the current state value and a function to update it.

Syntax

const [state, setState] = useState(initialValue);

- state → current value
- setState → updates value

Example

```
import React, { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);
```

```
  return (
    <div>
      <h2>{count}</h2>
      <button onClick={() => setCount(count + 1)}>
        Increment
      </button>
    </div>
  );
}
```

Working

- Initial value = 0
- Clicking button updates state
- Component re-renders automatically

2) useEffect() Hook

- This hook allows you to perform fetching data, setting up subscriptions, or manually manipulating the DOM in functional components.
- It takes a callback function as an argument, which is executed after the component renders.
- **Example**: useEffect(() => { fetchSomething(); }, []);

3) useContext() Hook

- This hook allows you to access values from a React Context, which is a way to share data between components without passing props down through the component tree.
- It's useful for sharing global state, themes, or other data that needs to be accessed by multiple components.
- **Example:** const { theme } = useContext(ThemeContext);

Advantages of Hooks

✓ Simple functional components
✓ Less code than classes
✓ Reusable logic
✓ Better readability
✓ No this keyword